

BatchedSRNNCell – Equations & Parameters

This document specifies the ODE system implemented by `BatchedSRNNCell` in `train_srnn/models/srnn_cell.py`. The model is the PyTorch successor to the MATLAB `SRNNModel` documented in `docs/EquationsParametersDocs/parameter_table.md` of the `ConnectivityAdaptation` repo. It has since added: a learnable global timescale, per-parameter log-gains, recurrent / input gain knobs, a **softplus** reparameterization that enforces non-negativity on weights (under Dale’s law) and on every time constant, an additive scalar / per-neuron split for the threshold and SFA offset, an optional zero-floor rescaling for the STD variable, and a battery of multiplicative ablation masks. Three readout modes (synaptic / rate / dendritic) and four ODE solvers (semi-implicit, explicit Euler, RK4, exponential Euler) share the same right-hand side.

The document is structured in two layers:

1. **Pure math** – the ODEs as you would write them in a paper, with every transform absorbed into named effective parameters.
2. **Implementation** – the same equations with the **softplus**, **exp**, masking, and additive splits made explicit, mapping symbol-for-symbol onto the saved checkpoint keys.

A complete tabulation of state initial conditions, parameter initial conditions, and parameter shapes follows.

Conventions. K is the number of ablation variants packed into one `BatchedSRNNCell` (single-cell mode drops the leading K axis), B is batch, T is sequence length, N is `num_units`, $n_E = \lfloor N/2 \rfloor$, $n_I = N - n_E$, D is `input_size`, and $\Delta t = h/\text{ode_unfolds}$ is the substep used inside each call to `forward`.

1. State variables

The cell carries five state components, packed flat as $[a_E | a_I | b_E | b_I | x]$:

symbol	shape (batched)	range	meaning
x_i	(K, B, N)	$\mathbb{R}$	dendritic potential of neuron i
$a_{i,k}^E$	$(K, B, n_E, n_{aE}^{\max})$	$[0, 1]$	k -th SFA tier of E neuron i
$a_{i,k}^I$	$(K, B, n_I, n_{aI}^{\max})$	$[0, 1]$	k -th SFA tier of I neuron i
b_i^E	(K, B, n_E)	$[0, 1]$	STD available-resource fraction (E)
b_i^I	(K, B, n_I)	$[0, 1]$	STD available-resource fraction (I)

Inactive ablation tiers (e.g. SFA off in a `srnn-no-adapt` variant) are zero-padded inside the packed tensor and zeroed at compute time via boolean masks (§5.7).

Dependent quantities (recomputed every step):

symbol	shape	meaning
r_i	(K, B, N)	firing rate, $r_i = \phi(x_i^{\text{eff}} - a_{0,i})$
$\tilde{b}_i$	(K, B, N)	effective synaptic gain (post-zero-floor, post-mask)
$\tilde{b}_i r_i$	(K, B, N)	synaptic output (presynaptic depression applied)
u_i	(K, B, N)	external drive, $u = W_{\text{in,eff}} u_{\text{ext}}$

2. Pure-math ODE system

For a single variant (drop the K axis), the network state evolves as

$$\tau_d \dot{x}_i = -x_i + u_i + \sum_{j=1}^N W_{ij} \tilde{b}_j r_j$$

$$r_i = \phi \left(x_i - a_{0,i} - \sum_{k=1}^{n_a^{(i)}} c_{i,k} a_{i,k} \right)$$

$$\tau_{a,(i,k)} \dot{a}_{i,k} = -a_{i,k} + r_i + c_{0,(i,k)}$$

$$\dot{b}_i = \frac{1 - b_i}{\tau_i^{\text{rec}}} - \frac{b_i r_i}{\tau_i^{\text{rel}}}$$

where $\tilde{b}_i = b_i$ in the simple case and $\tilde{b}_i$ is the zero-floor-rescaled version when `std_zero_floor=True` (see §5.6). The SFA tier index k ranges over $\{1, \dots, n_a^{(i)}\}$, where $n_a^{(i)} = n_{aE}$ if $i \leq n_E$ and $n_a^{(i)} = n_{aI}$ otherwise. Likewise $\tau_i^{\text{rec}}, \tau_i^{\text{rel}}$ refer to the E constants for $i \leq n_E$ and the I constants for $i > n_E$.

Dale’s law. When active, W has all-non-negative entries before sign assignment, with column sign $s_j \in \{+1, -1\}$ fixed by the neuron’s E/I identity ($s_j = +1$ for $j \leq n_E$, else -1). Equivalently, columns of W are sign-constrained: an E-source column is non-negative, an I-source column is non-positive.

External drive. The input map is $u = W_{\text{in,eff}} u_{\text{ext}}$ where $u_{\text{ext}} \in \mathbb{R}^D$ is the per-step exogenous input and $W_{\text{in,eff}} \in \mathbb{R}^{N \times D}$ is masked by the input-neuron partition (only a subset of rows is non-zero).

2.1 Vectorized form

Stack the per-neuron states into vectors and matrices: $\mathbf{x} \in \mathbb{R}^N$, $\mathbf{r} \in \mathbb{R}^N$, $\mathbf{a}_0 \in \mathbb{R}^N$, $\mathbf{b} \in \mathbb{R}^N$ (concatenation $[b^E; b^I]$), $\tau_d \in \mathbb{R}^N$, $\tau^{\text{rec}}, \tau^{\text{rel}} \in \mathbb{R}^N$. Stack the SFA state and parameters into matrices padded over a common tier dimension $K_a = \max(n_{aE}, n_{aI})$: $A, C, C_0, \mathcal{T}_a \in \mathbb{R}^{N \times K_a}$ (rows $1:n_E$ hold the E-side entries with the I columns zero-padded when $n_{aI} > n_{aE}$, and analogously for rows $n_E+1:N$). Let $\mathbf{1}_M$ denote the all-ones vector of length M , $\odot$ the Hadamard (elementwise) product, and $\oslash$ elementwise division. Then §2 reads

$$\tau_d \odot \dot{\mathbf{x}} = -\mathbf{x} + W(\tilde{\mathbf{b}} \odot \mathbf{r}) + \mathbf{u}, \quad \mathbf{u} = W_{\text{in}} \mathbf{u}_{\text{ext}}$$

$$\mathbf{r} = \phi(\mathbf{x} - \mathbf{a}_0 - (C \odot A) \mathbf{1}_{K_a})$$

$$\mathcal{T}_a \odot \dot{A} = -A + \mathbf{r} \mathbf{1}_{K_a}^\top + C_0$$

$$\dot{\mathbf{b}} = (\mathbf{1}_N - \mathbf{b}) \oslash \tau^{\text{rec}} - (\mathbf{b} \odot \mathbf{r}) \oslash \tau^{\text{rel}}.$$

The outer product $\mathbf{r} \mathbf{1}_{K_a}^\top$ broadcasts the firing rate across all SFA tiers. The row-sum $(C \odot A) \mathbf{1}_{K_a}$ collapses tiers to a single per-neuron SFA contribution. When the SFA or STD path is disabled for a side, the corresponding rows of $C, A, C_0, \mathcal{T}_a$ contain zeros (or $\tilde{\mathbf{b}} = \mathbf{1}$ for STD), implementing the ablation gate without changing the equation form.

2.2 Activation function ϕ

The activation is a hard sigmoid with quadratic-rounded corners that maps $\mathbb{R} \rightarrow [0, 1]$. With width parameter $S_a \in [0, 1]$ (default 0.9) and centre $c = S_c$ (default 0 in PyTorch – the threshold a_0 is applied externally before ϕ),

$$a = S_a/2, \quad k = \frac{1}{2(1-2a)}, \quad x_1 = c + a - 1, \quad x_2 = c - a, \quad x_3 = c + a, \quad x_4 = c + 1 - a.$$

$$\phi(z) = \begin{cases} 0 & z < x_1 \\ k(z - x_1)^2 & x_1 \leq z < x_2 \\ (z - c) + \frac{1}{2} & x_2 \leq z \leq x_3 \\ 1 - k(z - x_4)^2 & x_3 < z \leq x_4 \\ 1 & z > x_4. \end{cases}$$

The quadratic patches preserve C^1 continuity. With $S_c = 0$ the midpoint $\phi(0) = \frac{1}{2}$, and the threshold appears as $z = x_i^{\text{eff}} - a_{0,i}$ on the call site.

3. Expanded ODE system: global scaling surfaced

The effective parameters τ_d , $\tau_{a,(i,k)}$, $\tau_i^{\text{rec/rel}}$, $c_{(i,k)}$, W , W_{in} , a_0 , c_0 that appear in §2 each decompose into a product (or sum) of a global piece, a per-class piece, and a per-axis piece. This section rewrites the ODEs with that multi-level structure made explicit. The **softplus** / **exp** reparameterization that enforces non-negativity in code is *not* shown here (it is deferred to §5); every symbol below is just a non-negative scalar / vector / matrix.

Notation. Subscripts: τ_g is the global timescale (scalar per variant); the per-class scalars take the form $\tau_{\bullet}^{\text{group}}$ (timescales), $c_{\bullet}^{\text{group}}$ (SFA coupling), W^{group} and $W_{\text{in}}^{\text{group}}$ (recurrent / input gains) – one for each class $\bullet \in \{W, \text{in}, \tau_d, \tau_a^E, \tau_a^I, c^E, c^I, \tau_b^{\text{rec},E}, \tau_b^{\text{rel},E}, \tau_b^{\text{rec},I}, \tau_b^{\text{rel},I}\}$; the superscript 0 marks a per-axis base ($\tau_{d,i}^0$ is the per-neuron base, $\tau_{a,(i,k)}^0$ is the per-neuron / per-tier base, W^0 is the base recurrent matrix, etc.). W^0 already includes the binary connectivity mask and the Dale column-sign structure (this is how `_effective_W` is built and saved); $W^{0,\text{in}}$ already includes the input-neuron row mask. The threshold and SFA offset are sums: $a_0 = a_0^{\text{vec}} + \bar{a}_0$ and $c_{0,(i,k)} = c_{0,(i,k)}^{\text{vec}} + \bar{c}_0^{(i)}$, each with a per-neuron / per-tier vector and a per-variant additive scalar (for c_0 , the scalar is class-specific to E or I). The superscript (i) on a class symbol selects the E or I value according to whether i indexes an E or an I neuron.

3.1 Indexed form

$$(\tau_g \tau_d^{\text{group}} \tau_{d,i}^0) \dot{x}_i = -x_i + u_i + W^{\text{group}} \sum_{j=1}^N W_{ij}^0 \tilde{b}_j r_j$$

$$u_i = W_{\text{in}}^{\text{group}} \sum_{m=1}^D W_{im}^{0,\text{in}} u_{\text{ext},m}$$

$$r_i = \phi \left(x_i - (a_{0,i}^{\text{vec}} + \bar{a}_0) - c_{(i)}^{\text{group}} \sum_{k=1}^{n_a^{(i)}} c_{(i,k)}^0 a_{i,k} \right)$$

$$(\tau_g \tau_{a,(i)}^{\text{group}} \tau_{a,(i,k)}^0) \dot{a}_{i,k} = -a_{i,k} + r_i + (c_{0,(i,k)}^{\text{vec}} + \bar{c}_0^{(i)})$$

$$\dot{b}_i = \frac{1 - b_i}{\tau_g \tau_{b^{\text{rec}},(i)}^{\text{group}} \tau_i^{0,\text{rec}}} - \frac{b_i r_i}{\tau_g \tau_{b^{\text{rel}},(i)}^{\text{group}} \tau_i^{0,\text{rel}}}.$$

Time-rescaling intuition. Substituting $t' = t/\tau_g$ divides τ_g out of every LHS coefficient and out of the b -equation denominators, so τ_g acts as a uniform time-axis stretch – one knob to retune all four classes of timescales together. The per-class $\tau_{\bullet}^{\text{group}}$ then introduces relative drift between classes (e.g. τ_d shifting relative to τ_a).

3.2 Vectorized form

With the same stacking conventions as §2.1, define base counterparts: $\tau_d^0 \in \mathbb{R}_{\geq 0}^N$, $\mathcal{T}_a^0 \in \mathbb{R}_{\geq 0}^{N \times K_a}$, $\tau^{0,\text{rec}}, \tau^{0,\text{rel}} \in \mathbb{R}_{\geq 0}^N$, $C^0 \in \mathbb{R}_{\geq 0}^{N \times K_a}$, $W^0 \in \mathbb{R}^{N \times N}$, $W^{0,\text{in}} \in \mathbb{R}^{N \times D}$, $\mathbf{a}_0^{\text{vec}} \in \mathbb{R}^N$. Per-class scalars τ_a^{group} , $\mathbf{c}^{\text{group}}$, $\tau_{b^{\text{rec}}}^{\text{group}}$, $\tau_{b^{\text{rel}}}^{\text{group}} \in \mathbb{R}_{\geq 0}^N$ are the per-neuron lifts of the corresponding E / I scalars (E value on rows 1: n_E , I value on rows n_E+1 : N); $\bar{\mathbf{c}}_0 \in \mathbb{R}^N$ is the analogous lift of $\bar{c}_0^E, \bar{c}_0^I$.

$$(\tau_g \tau_d^{\text{group}} \tau_d^0) \odot \dot{\mathbf{x}} = -\mathbf{x} + W^{\text{group}} W^0 (\tilde{\mathbf{b}} \odot \mathbf{r}) + \mathbf{u}$$

$$\mathbf{u} = W_{\text{in}}^{\text{group}} W^{0,\text{in}} \mathbf{u}_{\text{ext}}$$

$$\mathbf{r} = \phi \left(\mathbf{x} - (\mathbf{a}_0^{\text{vec}} + \bar{a}_0 \mathbf{1}_N) - (\mathbf{c}^{\text{group}} \odot (C^0 \odot A) \mathbf{1}_{K_a}) \right)$$

$$(\tau_g \tau_a^{\text{group}} \mathbf{1}_{K_a}^\top \odot \mathcal{T}_a^0) \odot \dot{A} = -A + \mathbf{r} \mathbf{1}_{K_a}^\top + (C_0^{\text{vec}} + \bar{\mathbf{c}}_0 \mathbf{1}_{K_a}^\top)$$

$$\dot{\mathbf{b}} = (\mathbf{1}_N - \mathbf{b}) \oslash (\tau_g \tau_{b^{\text{rec}}}^{\text{group}} \odot \tau^{0,\text{rec}}) - (\mathbf{b} \odot \mathbf{r}) \oslash (\tau_g \tau_{b^{\text{rel}}}^{\text{group}} \odot \tau^{0,\text{rel}}).$$

The outer product $\tau_a^{\text{group}} \mathbf{1}_{K_a}^\top$ broadcasts the per-neuron class-gain across SFA tiers. The base matrices W^0 and $W^{0,\text{in}}$ already carry the sparsity / Dale-sign / input-mask structure; no extra mask factors are needed in the equations.

Going from §3 to §5.1-5.4 is a one-step substitution: each τ_*^0 or C^0 is replaced by σ^+ of a learnable log-vector, τ_g is replaced by $\sigma^+(\ell_{\tau_g})$, and each $\bullet^{\text{group}}$ is replaced by $e^{g_\bullet}$ for a learnable log-gain $g_\bullet$. That replacement produces exactly the formulas in §5.1-5.5.

4. Discretization (one forward step)

Each call to `forward` advances the state by `h` seconds in `ode_unfolds` substeps of size $\Delta t = h/\text{ode_unfolds}$. All four solvers share the continuous RHS in §2 (or, equivalently, the expanded form in §3, computed by `_batched_compute_rhs` after applying the transforms of §5); they differ only in how the state is updated.

Semi-implicit (default; `_batched_step_semi_implicit`). For each linear ODE $\tau \dot{y} = -y + S$ the update is the implicit Euler step

$$y^+ = \frac{y + \alpha S}{1 + \alpha}, \quad \alpha = \Delta t / \tau.$$

Applied to x , a , and (with a quasi-linearization) b :

$$\begin{aligned} x^+ &= \frac{x + \alpha_x (u + W_{\text{eff}}(\tilde{b} \odot r))}{1 + \alpha_x}, & \alpha_x &= \Delta t / \tau_d \\ a_{i,k}^+ &= \frac{a_{i,k} + \alpha_{a,(i,k)} (c_{0,(i,k)} + r_i)}{1 + \alpha_{a,(i,k)}}, & \alpha_{a,(i,k)} &= \Delta t / \tau_{a,(i,k)} \\ b_i^+ &= \frac{b_i + \Delta t / \tau_i^{\text{rec}}}{1 + \Delta t (1/\tau_i^{\text{rec}} + r_i/\tau_i^{\text{rel}})}, & b_i^+ &\leftarrow \text{clamp}(b_i^+, 0, 1). \end{aligned}$$

Explicit Euler (`_batched_step_explicit`). $y^+ = y + \Delta t \cdot \dot{y}$, b clamped to $[0, 1]$.

RK4 (`_batched_step_rk4`). Standard fourth-order Runge-Kutta on the joint state (x, a_E, a_I, b_E, b_I) , b clamped after the final combination.

Exponential Euler (`_batched_step_exponential`). For linear components,

$$x^+ = x e^{-\Delta t / \tau_d} + (1 - e^{-\Delta t / \tau_d}) (u + W_{\text{eff}}(\tilde{b} \odot r))$$

with the analogous closed form for a . The nonlinear b uses an explicit Euler step with clamp.

5. Implementation: surfaced transforms

In code, every parameter is stored in a *raw* form and transformed at read time so that effective time constants and synaptic weights remain non-negative throughout training. Let $\sigma^+(\cdot) = \log(1 + \exp(\cdot))$ denote the softplus and $g \mapsto e^g$ denote the log-gain. Subscripts $(K, \cdot, \cdot)$ are dropped on the right-hand sides when shapes are unambiguous.

5.1 Recurrent weight

$$W_{\text{eff}} = g_W \cdot \left[m_{\text{Dale}} \cdot (s \otimes \mathbf{1}) \odot \sigma^+(W_{\text{raw}}) + (1 - m_{\text{Dale}}) \cdot W_{\text{raw}} \right] \odot S_{\text{sparsity}}$$

where $s \in \{+1, -1\}^N$ is the per-column Dale sign (`dales_signs`), $m_{\text{Dale}} \in \{0, 1\}$ is the per-variant Dale flag (`dales_mask`), $S_{\text{sparsity}} \in \{0, 1\}^{N \times N}$ is the binary connectivity mask from the RMT initializer, and $g_W \geq 0$ is the per-variant scalar `W_raw_gain` (init 1). The outer product $s \otimes \mathbf{1}$ broadcasts the sign vector across rows so that the column-sign constraint applies regardless of the post-synaptic neuron.

When `echo=True` for a variant, a backward hook zeros the gradient on W_{raw} (so the recurrent structure is frozen at its RMT initialization), but g_W remains trainable – the variant retains a single learnable spectral-radius knob.

5.2 Input weight

$$W_{\text{in,eff}} = g_{\text{in}} \cdot W_{\text{in}} \odot M_{\text{in}}$$

with g_{in} (`W_in_gain`, scalar per variant, init 1) and a binary row mask $M_{\text{in}} \in \{0, 1\}^{N \times 1}$ that restricts external drive to the input-partition neurons.

5.3 Threshold and SFA offset

$$a_{0,i} = a_{0,i}^{\text{vec}} + a_0^{\text{scalar}}, \quad c_{0,(i,k)} = c_{0,(i,k)}^{\text{vec}} + c_{0,(.)}^{\text{scalar}}.$$

Both are direct (no `softplus`); the per-neuron vector and the per-variant additive scalar are independent learnables. The `per_neuron=False` ablation freezes the vector via a gradient-zeroing hook so only the scalar moves; `per_neuron=True` lets both train, giving a “shared direction + per-neuron deviation” parameterization.

5.4 Time constants

Every effective time constant is the product of the global timescale, a per-parameter exponentiated log-gain, and a per-axis softplus of a log-vec. Concretely,

$$\tau_g = \sigma^+(\ell_{\tau_g}) \quad (\text{scalar per variant})$$

$$\tau_{d,i} = \tau_g \cdot e^{g_{\tau_d}} \cdot \sigma^+(\ell_{\tau_{d,i}}^{\text{vec}})$$

$$\tau_{a^E,(i,k)} = \tau_g \cdot e^{g_{\tau_a^E}} \cdot \sigma^+(\ell_{\tau_a^E,(i,k)}^{\text{vec}}) \quad (\text{same form for } I)$$

$$\tau_i^{\text{rec},E} = \tau_g \cdot e^{g_{\tau_b^{\text{rec},E}}} \cdot \sigma^+(\ell_{\tau_b^{\text{rec},E},i}^{\text{vec}}) \quad (\text{same for } \tau^{\text{rel}}, \text{ and for } I)$$

The shared τ_g stretches every effective time constant in lockstep (useful for matching the data sample-rate without re-initializing individual taus); the per-parameter e^g adds a class-level free axis on top of the per-axis vec.

5.5 SFA coupling

$$c_{(i,k)}^E = e^{g_{c^E}} \cdot \sigma^+(\ell_{c^E,(i,k)}^{\text{vec}}), \quad c_{(i,k)}^I = e^{g_{c^I}} \cdot \sigma^+(\ell_{c^I,(i,k)}^{\text{vec}}).$$

Effective coupling is non-negative by construction.

5.6 STD zero-floor rescaling

By default (`std_zero_floor=True`) the synaptic gain is rescaled so that the steady-state minimum of b_i at maximal firing maps to 0:

$$b_i^{\text{min}} = \frac{\tau_i^{\text{rel}}}{\tau_i^{\text{rec}} + \tau_i^{\text{rel}}}, \quad \tilde{b}_i = \frac{b_i - b_i^{\text{min}}}{1 - b_i^{\text{min}}}.$$

When `std_zero_floor=False`, $\tilde{b}_i = b_i$. This rescaling is applied in `_compute_b_full` and reused identically by `get_diagnostics`, both solver step functions, and the readout path.

The dynamics in §2 are unchanged – only the *post-state* synaptic output uses $\tilde{b}$; the ODE $\dot{b}_i$ still uses the raw b_i .

mask	shape	role
------	-------	------

5.7 Ablation masks

mask	shape	role
dales_mask	$(K, 1, 1)$	1 iff Dale's-law variant; selects $\sigma^+ \cdot s$ vs raw branch in W_{eff}
sfa_E_mask	$(K, 1, n_{aE}^{\max})$	1 on active E SFA tiers; multiplies c^E before the sum in x^{eff} and the RHS of $\dot{a}^E$
sfa_I_mask	$(K, 1, n_{aI}^{\max})$	analogous for I
std_E_mask	$(K, 1)$	1 iff E-side STD active; blends $\tilde{b}^E$ with 1
std_I_mask	$(K, 1)$	analogous for I
std_zero_floor_mask	$(K, 1, 1)$	per-variant flag enabling the rescale of §5.6
echo_flags	$(K, 1, 1)$	1 iff echo variant; companion <code>_echo_grad_mask = 1 - echo_flags</code> is multiplied into incoming gradients on W_{raw}
skip_flags	$(K,)$	1 iff skip variant; selects the residual $+x_{\text{readout}}$ path in <code>SequenceModel._readout_one</code>
_a_0_vec_mask, _log_tau_d_vec_mask, _log_tau_a_{E,I}_vec_mask, _log_c_{E,I}_vec_mask, _c_0_{E,I}_vec_mask, _log_tau_b_{rec,rel}_{E,I}_vec_mask	$(K, 1, \dots)$	gradient-zeroing masks: per-neuron vec components are frozen for <code>per_neuron=False</code> variants
readout_ids	$(K,)$ in $\{0, 1, 2\}$	per-variant readout selector (0 synaptic, 1 rate, 2 dendritic)

The ablation masks are buffers (not parameters), set once at construction from the per-variant `SRNNConfig`. See `KnownIssues.md` §1 for the per-neuron / `BatchedSRNNCell` gradient-mask story – the per-neuron vec parameters are *always* allocated at full $(K, N, \dots)$ shape regardless of `per_neuron`, but their gradients are zeroed for non-per-neuron variants so they remain at the (uniform) init.

5.8 Readout

Per-variant readout mode selects one of three signals:

$$y_t = \mathbb{1}[\text{readout} = 0] \tilde{b} r + \mathbb{1}[\text{readout} = 1] r + \mathbb{1}[\text{readout} = 2] x$$

evaluated at the final substep of the forward call. `SequenceModel` applies an output-neuron mask, a per-variant linear head, and (for `skip=True` variants) adds the residual $\alpha_k x_{\text{readout}}$ where α_k is `skip_flags[k]`. Skip is only legal when `input_size == output_size` and is implemented in `train_srnn/models/sequence_model.py:_readout_one`.

6. Initial conditions

6.1 State ICs (per call to `init_state`)

state	raw init	effective init at $t = 0$	shape	source
x	$0.1 \cdot \mathcal{N}(0, 1)$	same (no transform)	(K, B, N)	<code>init_state</code> line 1304
a^E	0	0	$(K, B, n_E, n_{aE}^{\max})$	<code>init_state</code>
a^I	0	0	$(K, B, n_I, n_{aI}^{\max})$	<code>init_state</code>

state	raw init	effective init at $t = 0$	shape	source
b^E	1	1 if STD on; 1 from <code>std_E_mask</code> blend if STD off	(K, B, n_E)	<code>init_state</code>
b^I	1	1	(K, B, n_I)	<code>init_state</code>

When `TrainableIC` is enabled, the cell's `init_state` output is overwritten by a learnable $\hat{x}_0$ buffer that has been pre-burned in for `compute_burn_in` steps; the SFA and STD components remain at 0 and 1. See `train_srn/utls/trainable_ic.py`.

6.2 Parameter ICs (raw $\rightarrow$ effective)

The transforms in §5 give the effective values below at training step 0. $\sigma^{+,-1}$ denotes inverse-softplus.

effective param	raw param init	effective value at $t = 0$
g_W (<code>W_raw_gain</code>)	1	1
g_{in} (<code>W_in_gain</code>)	1	1
W_{raw}	RMT-initialized real matrix (see <code>RMTMatrix</code>)	—
W_{in}	$\mathcal{N}(0, 0.1^2)$	—
$a_{0,i}^{vec}$	0.35	0.35 (since scalar = 0)
a_0^{scalar}	0	—
τ_g	$\ell_{\tau_g} = \sigma^{+,-1}(\tau_g^{init})$, default $\tau_g^{init} = 1$	1 s
$\tau_{d,i}$	$\ell_{\tau_d}^{vec} = \sigma^{+,-1}(0.1)$, $g_{\tau_d} = 0$	0.1 s
$\tau_{a,(i,k)}$, $n_a = 1$	$\ell_{\tau_d}^{vec} = \sigma^{+,-1}(1.0)$, $g = 0$	1 s
$\tau_{a,(i,k)}$, $n_a \geq 2$	linear interp in inv-softplus space between $\sigma^{+,-1}(0.25)$ and $\sigma^{+,-1}(10)$ over $k = 0, \dots, n_a - 1$, $g = 0$	$\{0.25, \dots, 10\}$ s logspaced
$c_{(i,k)}$	$\ell_c^{vec} = \sigma^{+,-1}(0.05)$, $g_c = 0$	0.05
$c_{0,(i,k)}^{vec}$, scalar	0 each	0
τ_i^{rec} (E or I)	$\sigma^{+,-1}(1.0)$, $g = 0$	1 s
τ_i^{rel} (E or I)	$\sigma^{+,-1}(0.25)$, $g = 0$	0.25 s

Per-variant readout linear head is `nn.Linear` initialised with `kaiming_uniform_`; $g_{\tau_d}, g_{\tau_a^E, I}, g_{c^E, I}, g_{\tau_b^E, I}$ are all log-gains initialised to 0 (multiplicative identity).

7. Complete parameter table

Every learnable parameter and structural buffer of `BatchedSRNNCell` plus the `SequenceModel` readout head appears below.

7.1 Learnable parameters (`nn.Parameter`)

math symbol	code key	shape	init	transform $\rightarrow$ effective
W_{raw}	<code>cell.W_raw</code>	(K, N, N)	RMT init	W_{eff} via §5.1
W_{in}	<code>cell.W_in</code>	(K, N, D)	$\mathcal{N}(0, 0.01)$	$W_{in,eff}$ via §5.2
g_W	<code>cell.W_raw_gain</code>	$(K,)$	1	direct
g_{in}	<code>cell.W_in_gain</code>	$(K,)$	1	direct
a_0^{vec}	<code>cell.a_0_vec</code>	(K, N)	0.35	$a_0 = a_0^{vec} + a_0^{scalar}$
a_0^{scalar}	<code>cell.a_0_scalar</code>	$(K,)$	0	as above
ℓ_{τ_g}	<code>cell.log_tau_global</code>	$(K,)$	$\sigma^{+,-1}(1)$	$\tau_g = \sigma^+(\ell_{\tau_g})$
$\ell_{\tau_d}^{vec}$	<code>cell.log_tau_d_vec</code>	(K, N)	$\sigma^{+,-1}(0.1)$	combined per §5.4
g_{τ_d}	<code>cell.log_tau_d_gain</code>	$(K,)$	0	$e^{g_{\tau_d}}$ multiplier
$\ell_{\tau_a^E}^{vec}$	<code>cell.log_tau_a_E_vec</code>	(K, n_E, n_{aE}^{max})	logspaced (see 6.2)	combined per §5.4
$g_{\tau_a^E}$	<code>cell.log_tau_a_E_gain</code>	$(K,)$	0	e^g multiplier
$\ell_{c^E}^{vec}$	<code>cell.log_c_E_vec</code>	(K, n_E, n_{aE}^{max})	$\sigma^{+,-1}(0.05)$	c^E via §5.5

math symbol	code key	shape	init	transform → effective
g_c^E	cell.log_c_E_gain	$(K,)$	0	e^g multiplier
$c_{0,E}^{\text{vec}}$	cell.c_0_E_vec	$(K, n_E, n_{aE}^{\text{max}})$	0	$c_{0,E} = \text{vec} + \text{scalar}$
$c_{0,E}^{\text{scalar}}$	cell.c_0_E_scalar	$(K,)$	0	as above
$\ell_{\tau_a^I}^{\text{vec}}, g_{\tau_a^I}, \ell_{c^I}^{\text{vec}}, g_{c^I}, c_{0,I}^{\text{vec}}, c_{0,I}^{\text{scalar}}$	cell.log_tau_a_I_vec, I-side analogues		as E side	as E side
	cell.log_tau_a_I_gain,			
	cell.log_c_I_vec,			
	cell.log_c_I_gain,			
	cell.c_0_I_vec,			
	cell.c_0_I_scalar			
$\ell_{\tau_b^{\text{rec},E}}^{\text{vec}}$	cell.log_tau_b_rec_E_vec	(K, n_E)	$\sigma^{+,-1}(1)$	$\tau^{\text{rec},E}$ per §5.4
$g_{\tau_b^{\text{rec},E}}$	cell.log_tau_b_rec_E_gain	$(K,)$	0	e^g multiplier
$\ell_{\tau_b^{\text{rel},E}}^{\text{vec}}$	cell.log_tau_b_rel_E_vec	(K, n_E)	$\sigma^{+,-1}(0.25)$	$\tau^{\text{rel},E}$ per §5.4
$g_{\tau_b^{\text{rel},E}}$	cell.log_tau_b_rel_E_gain	$(K,)$	0	e^g multiplier
$\tau^{\text{rec},I}, \tau^{\text{rel},I}$ params	cell.log_tau_b_rec_I_vec, I-side analogues		as E side	as E side
	cell.log_tau_b_rec_I_gain,			
	cell.log_tau_b_rel_I_vec,			
	cell.log_tau_b_rel_I_gain			
W_{out}	readout_weight	(K, O, E)	kaiming_uniform_(a=sqrt(5))	linear head
b_{out}	readout_bias	(K, O)	0	direct
$\hat{x}_0$	ic.ic	(K, N) or $(N,)$	burn-in result if TrainableIC	replaces x_0

For variants with $n_{a,E} = 0$, $n_{a,I} = 0$, $n_{b,E} = 0$, or $n_{b,I} = 0$, the entire E- or I-side SFA / STD parameter block is **None** rather than zero – the parameters are not constructed.

7.2 Buffers (non-trainable, set at construction)

code key	shape	role
cell.sparsity_masks	(K, N, N)	S_{sparsity} in §5.1
cell.dales_signs	(K, N)	column signs s in §5.1
cell.W_in_mask	$(1, N, 1)$	input-neuron partition M_{in}
cell.dales_mask	$(K, 1, 1)$	per-variant Dale flag
cell.echo_flags	$(K, 1, 1)$	per-variant echo flag
cell._echo_grad_mask	$(K, 1, 1)$	1 – echo, multiplied into ∇W_{raw}
cell.skip_flags	$(K,)$	per-variant skip flag
cell.sfa_E_mask / cell.sfa_I_mask	$(K, 1, n_a^{\text{max}})$	active SFA tiers
cell.std_E_mask / cell.std_I_mask	$(K, 1)$	STD on/off
cell.std_zero_floor_mask	$(K, 1, 1)$	enable rescale of §5.6 (non-persistent)
cell.readout_ids	$(K,)$	$\{0, 1, 2\}$ readout selector
cell._a_0_vec_mask,	$(K, 1, \dots)$	gradient-zero mask for non-per-neuron
cell._log_tau_d_vec_mask,		variants
cell._log_tau_a_{E,I}_vec_mask,		
cell._log_c_{E,I}_vec_mask,		
cell._c_0_{E,I}_vec_mask,		
cell._log_tau_b_{rec,rel}_{E,I}_vec_mask		

7.3 Compile-time configuration (per SRNNConfig)

These do not appear in the state dict; they shape construction.

field	default	meaning
num_units	32	N
dales	True	sets dales_mask
$n_{a,E}, n_{a,I}$	3, 3	SFA tier count per side (0 disables)

field	default	meaning
n_b_E, n_b_I	1, 1	STD on/off per side
per_neuron	False	toggle gradient-zero hook on *_vec params
echo	False	freeze W_{raw} gradient
skip	False	residual $+x_{\text{readout}}$ at output
solver	"semi_implicit"	one of semi_implicit, explicit, rk4, exponential
h	0.02	seconds simulated per forward call
ode_unfolds	1	substep count; $\Delta t = h/\text{ode_unfolds}$
readout	"synaptic"	maps to readout_ids $\in \{0, 1, 2\}$
tau_global_init	1.0	initial τ_g value (in seconds)
std_zero_floor	True	enable §5.6 rescale

All variants packed into one BatchedSRNNCell must agree on num_units, solver, h, and ode_unfolds.

8. Ablation presets

The presets in SRNN_PRESETS (train_srnn/models/srnn_cell.py:126) map a name to which mechanisms are active. The columns marked “.” are at default (non-zero, non-frozen).

preset	n_{aE}	n_{aI}	n_{bE}	n_{bI}	dales	echo	skip	per-neuron
srnn	3	3	1	1	yes	no	no	no
srnn-per-neuron	.	.	.	.	yes	no	no	yes
srnn-echo	.	.	.	.	yes	yes	no	no
srnn-no-adapt	0	0	0	0	yes	no	no	no
srnn-no-adapt-no-dales	0	0	0	0	no	no	no	no
srnn-sfa-only	1	1	0	0	yes	no	no	no
srnn-std-only	0	0	1	1	yes	no	no	no
srnn-E-only	3	0	1	0	yes	no	no	no
srnn-sfa-e-only	3	0	0	0	yes	no	no	no
srnn-std-e-only	0	0	1	0	yes	no	no	no
srnn-e-only-echo	3	0	1	0	yes	yes	no	no
srnn-e-only-per-neuron	3	0	1	0	yes	no	no	yes
srnn-e-only-skip	3	0	1	0	yes	no	yes	no
srnn-e-only-skip-per-neuron	3	0	1	0	yes	no	yes	yes
srnn-e-only-skip-echo	3	0	1	0	yes	yes	yes	no
srnn-skip	3	3	1	1	yes	no	yes	no

preset	n_{aE}	n_{aI}	n_{bE}	n_{bI}	dales	echo	skip	per-neuron
srnn-no-dales-skip	.	.	.	.	no	no	yes	no
srnn-no-adapt-no-dales-skip	0	0	0	0	no	no	yes	no
srnn-sfa-e-only-skip	3	0	0	0	yes	no	yes	no
srnn-std-e-only-skip	0	0	1	0	yes	no	yes	no
srnn-sfa-e-only-per-neuron	3	0	0	0	yes	no	no	yes
srnn-std-e-only-per-neuron	0	0	1	0	yes	no	no	yes
srnn-sfa-e-only-skip-per-neuron	3	0	0	0	yes	no	yes	yes
srnn-std-e-only-skip-per-neuron	0	0	1	0	yes	no	yes	yes
srnn-multi-sfa	2	2	1	1	yes	no	no	no
srnn-multi-sfa-E	2	0	1	0	yes	no	no	no
srnn-no-dales	.	.	.	.	no	no	no	no
srnn-explicit	.	.	.	.	yes (solver=explicit)	no	no	no
srnn-rk4	.	.	.	.	yes (solver=rk4)	no	no	no

Setting any of $n_{a\cdot}$ or $n_{b\cdot}$ to 0 removes the corresponding terms from §2: with $n_{aE} = 0$ the $c_{i,k} a_{i,k}$ sum in r vanishes for E neurons and the $\dot{a}^E$ equation is not integrated. With $n_{bE} = 0$ the $\dot{b}^E$ equation drops and $\tilde{b}_i^E = 1$ identically.

9. Cross-references

- `FullModel.md` – exhaustive, code-level companion spec, including `SequenceModel` plumbing (output masking, BPTT detaching, trainable IC burn-in, autoregressive teacher-forcing window).
- `/Users/richner.thomas/Desktop/local_code/ConnectivityAdaptation/docs/EquationsParametersDocs/parameter_table.md` – MATLAB ancestor model. Same backbone ODEs (§2), without the `softplus` / log-gain / ablation-mask machinery.
- `KnownIssues.md` §1 – per-neuron / `BatchedSRNNCell` gradient-mask interaction (`*_vec` parameters always allocated at full per-neuron shape).
- `scripts/postprocess.py` (`effective_taus`, `effective_W`, `effective_W_in`, `effective_c`, `effective_a_0`, `effective_c_0`) – canonical implementations that recover the effective values of §5 from a saved `state_dict`. Produces the per-variant `param_table.txt` which is the empirical sibling of §7 above.